

RaceDay — Section B: API Endpoint Plan

Part 1 Portfolio of Evidence — Planning document.
README.

Total endpoints: 27 — Authentication (4), Organiser Profile (2), Participant Profile (2), Venues (2), Events (5), Categories (4), Enrolments (4), Results (4)

Note: Users table split into two fully separate tables — Organisers and Participants — per revised ERD. No shared login/register/profile endpoints.

HTTP Method	Route	Description	Role Required	Request Body	Expected Response
POST	/api/auth/organiser/register	Creates a new Organiser account.	None (public)	{ fullName, email, password, phoneNumber }	201 Created — organiser object + token; 400 Bad Request; 409 Conflict — email already registered
POST	/api/auth/participant/register	Creates a new Participant account.	None (public)	{ fullName, email, password, phoneNumber, dateOfBirth }	201 Created — participant object + token; 400 Bad Request; 409 Conflict — email already registered
POST	/api/auth/organiser/login	Authenticates an Organiser and returns a token.	None (public)	{ email, password }	200 OK — token; 401 Unauthorized — invalid credentials
POST	/api/auth/participant/login	Authenticates a Participant and returns a token.	None (public)	{ email, password }	200 OK — token; 401 Unauthorized — invalid credentials
GET	/api/organisers/me	Returns the profile of the currently logged-in Organiser.	Organiser	None	200 OK — organiser profile; 401 Unauthorized
PUT	/api/organisers/me	Updates the currently logged-in Organiser's profile.	Organiser	{ fullName, phoneNumber }	200 OK — updated profile; 400 Bad Request; 401 Unauthorized
GET	/api/participants/me	Returns the profile of the currently logged-in Participant.	Participant	None	200 OK — participant profile; 401 Unauthorized
PUT	/api/participants/me	Updates the currently logged-in Participant's profile.	Participant	{ fullName, phoneNumber, dateOfBirth }	200 OK — updated profile; 400 Bad Request; 401 Unauthorized
GET	/api/venues	Returns a list of all venues, so an Organiser can pick one when creating an event.	Any (logged in)	None	200 OK — array of venue objects
POST	/api/venues	Creates a new venue (e.g. a new city/location not yet in the system).	Organiser	{ venueName, addressLine, city, province, latitude, longitude }	201 Created — new venue object; 400 Bad Request; 401 Unauthorized; 403 Forbidden — not an Organiser
GET	/api/events	Returns a list of all events, so anyone can browse upcoming races.	None (public)	None	200 OK — array of event objects
GET	/api/events/{id}	Returns the full details of one specific event.	None (public)	None	200 OK — event object; 404 Not Found — event doesn't exist
POST	/api/events	Creates a new event, capturing name, description, date, location, distance, and event type.	Organiser	{ eventName, description, eventDate, startTime, venueId, distanceKM, eventType }	201 Created — new event object; 400 Bad Request; 401 Unauthorized; 403 Forbidden — logged in but not an Organiser
PUT	/api/events/{id}	Updates an existing event's details. Only the Organiser who created it may edit it.	Organiser	{ eventName, description, eventDate, startTime, venueId, distanceKM, eventType }	200 OK — updated event; 400 Bad Request; 401 Unauthorized; 403 Forbidden — not this event's owner; 404 Not Found
DELETE	/api/events/{id}	Deletes an event. Only the Organiser who created it may delete it.	Organiser	None	204 No Content — deleted successfully; 401 Unauthorized; 403 Forbidden; 404 Not Found
GET	/api/events/{id}/categories	Returns all categories available for a specific event (e.g. Under 20, 10km, 21km).	None (public)	None	200 OK — array of category objects; 404 Not Found — event doesn't exist
POST	/api/events/{id}/categories	Adds a new age/distance category to a specific event.	Organiser	{ categoryName, distanceKM, minAge, entryFee }	201 Created — new category object; 400 Bad Request; 401 Unauthorized; 403 Forbidden; 404 Not Found
PUT	/api/categories/{id}	Updates an existing category's details.	Organiser	{ categoryName, distanceKM, minAge, entryFee }	200 OK — updated category; 400 Bad Request; 401 Unauthorized; 403 Forbidden;

					404 Not Found
DELETE	/api/categories/{id}	Deletes a category from an event.	Organiser	None	204 No Content; 401 Unauthorized; 403 Forbidden; 404 Not Found
POST	/api/enrolments	Enters the logged-in Participant into an event by selecting a category. Records the link between Participant, event, and category.	Participant	{ categoryId }	201 Created — new enrolment object with bib number; 400 Bad Request; 401 Unauthorized; 403 Forbidden — not a Participant; 409 Conflict — already enrolled in this category
GET	/api/enrolments/mine	Returns all enrolments belonging to the logged-in Participant.	Participant	None	200 OK — array of the participant's own enrolments; 401 Unauthorized
DELETE	/api/enrolments/{id}	Cancels the logged-in Participant's own enrolment.	Participant	None	204 No Content; 401 Unauthorized; 403 Forbidden — not this participant's enrolment; 404 Not Found
GET	/api/events/{id}/enrolments	Returns all enrolments for a specific event, so the Organiser can see who has entered.	Organiser	None	200 OK — array of enrolment objects; 401 Unauthorized; 403 Forbidden — not this event's owner; 404 Not Found
POST	/api/results	Captures a finish time (or distance covered, for DNF) and finishing position for a specific enrolment, after the event.	Organiser	{ enrolmentId, finishTime, distanceCoveredKM, position, status }	201 Created — new result object; 400 Bad Request; 401 Unauthorized; 403 Forbidden; 404 Not Found — enrolment doesn't exist; 409 Conflict — result already captured for this enrolment
PUT	/api/results/{id}	Updates a previously captured result (e.g. correcting a time or position).	Organiser	{ finishTime, distanceCoveredKM, position, status }	200 OK — updated result; 400 Bad Request; 401 Unauthorized; 403 Forbidden; 404 Not Found
GET	/api/results/mine	Returns all of the logged-in Participant's own results across all events they've completed.	Participant	None	200 OK — array of the participant's own results; 401 Unauthorized
GET	/api/events/{id}/results	Returns all results for a specific event, so the Organiser can see the full results list.	Organiser	None	200 OK — array of result objects; 401 Unauthorized; 403 Forbidden; 404 Not Found

Design Notes & References

- **Split Auth/Profile endpoints:** because Organisers and Participants are now two fully separate database tables (not one Users table with a Role column), registration, login, and profile endpoints are duplicated per role rather than shared — e.g. /api/auth/organiser/login vs /api/auth/participant/login. This keeps the two user types structurally and behaviourally separate end-to-end, from database to API.
- **/me routes** ensure a logged-in user can only ever act on their own record — never someone else's by guessing an ID. See Microsoft REST API Guidelines, "Collection URL Patterns" (github.com/microsoft/api-guidelines).
- **Token-based login** reflects standard JWT authentication in ASP.NET Core; each token will carry a claim identifying whether it belongs to an Organiser or a Participant. See Microsoft Learn, "Authentication and authorization in ASP.NET Core" (learn.microsoft.com/aspnet/core/security/).
- **HTTP status code usage** (200, 201, 204, 400, 401, 403, 404, 409) follows the standard definitions in MDN Web Docs, "HTTP response status codes" (developer.mozilla.org/en-US/docs/Web/HTTP/Status).
- **401 vs 403 distinction:** 401 means not authenticated at all (no/invalid token); 403 means authenticated but not permitted (wrong role/table, or not the resource owner). Applied consistently across every protected endpoint above. See OWASP REST Security Cheat Sheet (cheatsheetseries.owasp.org/cheatsheets/REST_Security_Cheat_Sheet.html).
- **Nested routes** (e.g. /api/events/{id}/categories, /api/events/{id}/enrolments) are used only where the child cannot exist without its parent, reflecting the FK relationships in the Section A ERD. Flat routes (e.g. /api/categories/{id}) are used once a resource's own ID is enough to identify it uniquely.
- **409 Conflict** on enrolment and result creation directly reflects the UNIQUE constraints in the Section C SQL script (UNIQUE(ParticipantID, CategoryID) on Enrolments; UNIQUE(EnrolmentID) on Results) — the API plan matches the database design exactly.
- **Venues endpoints** were added beyond the literal functional requirements list because the ERD's Events.VenueID foreign key requires a way to view and create venues.
- **Distance on Events:** distanceKM was added to the Events request/response bodies (nullable) to satisfy the functional requirement that each event capture a distance, in addition to the per-category distanceKM already defined in Categories.